BiteFixes Enterprise Architecture Manual

Volume 08

Event-Driven Architecture Specification (EDAS)

Document ID: BFA-008

Version: 1.0

Status: Draft

Language: English

Architecture Level: Enterprise

Table of Contents

1. Introduction

2. Event Philosophy

3. Event Architecture

4. Event Types

5. Event Lifecycle

6. Domain Events

7. Integration Events

8. Event Bus

9. Event Consumers

10. Event Producers

11. Event Store

12. Event Versioning

13. Event Security

14. Event Reliability

15. Event Monitoring

16. Event Replay

17. Event Governance

18. Event Catalog

19. Future Evolution

Chapter 1

Introduction

The Event-Driven Architecture enables independent communication between business domains.

Instead of calling each other directly, domains publish and subscribe to events.

This reduces coupling, improves scalability, and simplifies future integrations.

Chapter 2

Event Philosophy

A business event represents something that has already happened.

Examples:

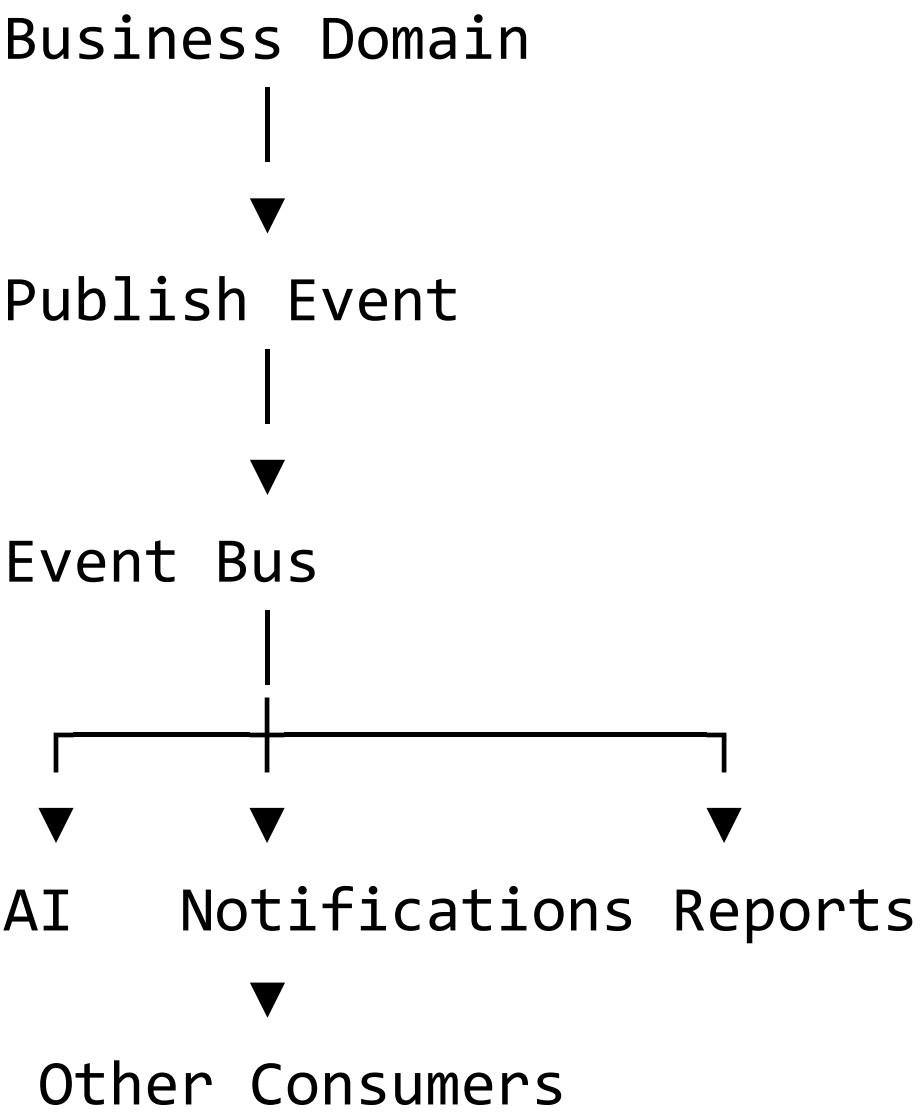
- Customer Registered
- Ticket Created
- Device Delivered
- Invoice Paid
- Knowledge Published

Events describe facts.

They never describe intentions or commands.

Chapter 3

Event Architecture



The publisher does not know who consumes the event.

Chapter 4

Event Types

The platform recognizes three primary event categories:

Domain Events

Internal business facts.

Example:

`TicketClosed`

Integration Events

Shared with external systems.

Example:

`InvoiceIssued`

System Events

Platform operational events.

Example:

```
UserLoggedIn
```

Chapter 5

Event Lifecycle

Every event follows the same lifecycle.

```
Raised

↓

Validated

↓

Published

↓

Consumed

↓
```

```
Archived
```

Events are immutable.

Chapter 6

Domain Events

Examples:

```
CompanyCreated

CustomerRegistered

DeviceRegistered

TicketOpened

TicketClosed

KnowledgePublished

InventoryReserved
```

`EstimateApproved`

Every important business action should emit an event.

Chapter 7

Integration Events

Integration events notify external platforms.

Examples:

`InvoiceGenerated`

`PaymentReceived`

`ShipmentCreated`

`CRMUpdated`

`WebhookReceived`

These events cross system boundaries.

Chapter 8

Event Bus

The Event Bus is responsible for routing events.

Responsibilities include:

- Delivery
- Routing
- Retry
- Ordering (when required)
- Filtering
- Monitoring

The Event Bus does not implement business logic.

Chapter 9

Event Producers

Any domain can publish events.

Example:

```
Ticket Domain

↓

TicketClosed

↓

Event Bus
```

The producer is responsible only for publishing.

Chapter 10

Event Consumers

Consumers subscribe to events of interest.

Example:

```
TicketClosed

↓

Notification Service

↓

Analytics Service

↓

Warranty Service

↓

Bitey AI
```

↓

```
Audit Service
```

Consumers remain independent from each other.

Chapter 11

Event Structure

Every event contains standardized metadata.

```
Event ID

Event Name

Version

Timestamp

Company ID

Correlation ID
```

`Causation ID`

`Producer`

`Payload`

`Schema Version`

This guarantees consistency across all domains.

Chapter 12

Correlation and Causation

To trace complete business processes:

Correlation ID

Groups all events belonging to the same business transaction.

Example:

```
Customer Request
↓
Ticket Created
↓
Estimate Generated
↓
Invoice Issued
↓
Payment Received
```

All share the same Correlation ID.

Causation ID

Identifies the event that triggered another event.

This enables complete audit trails.

Chapter 13

Event Store

Every published event should be stored.

The Event Store provides:

- Audit history
- Replay
- Analytics
- Debugging
- Compliance

The Event Store is append-only.

Events are never modified.

Chapter 14

Reliability

The event system guarantees:

- At-least-once delivery
- Retry on transient failures
- Dead-letter queue for unrecoverable events
- Idempotent consumers

Consumers must tolerate duplicate events.

Chapter 15

Event Replay

Historical events can be replayed.

Use cases include:

- Rebuilding projections

- Recovering analytics
- Testing new consumers
- Reprocessing AI models

Replay never changes historical events.

Chapter 16

Event Security

Every event must be:

- Authenticated
- Authorized
- Validated
- Logged

Sensitive information must be encrypted or omitted.

Events should contain only the minimum required data.

Chapter 17

Event Monitoring

The platform tracks:

- Published events
- Failed events
- Retry count
- Processing time
- Consumer latency
- Queue size
- Dead-letter messages

These metrics feed operational dashboards.

Chapter 18

Event Catalog

The Event Catalog documents every official event.

Example:

Event	Producer	Consumers
CustomerRegistered	Customer Domain	AI, Analytics, CRM
TicketOpened	Ticket Domain	Notifications, AI
TicketClosed	Ticket Domain	Warranty, Reports
PaymentReceived	Financial Domain	Ticket, Analytics

The catalog is maintained alongside the Business Rules Catalog.

Chapter 19

Event Governance

Before creating a new event, architects must evaluate:

- Is an existing event sufficient?
- Is the event reusable?
- Is the payload minimal?
- Does it expose sensitive data?
- Does it require versioning?

Events become part of the platform contract and should evolve carefully.

Chapter 20

Enterprise Vision

The Event-Driven Architecture transforms BiteFixes into an extensible platform.

Future modules can subscribe to existing events without modifying current domains.

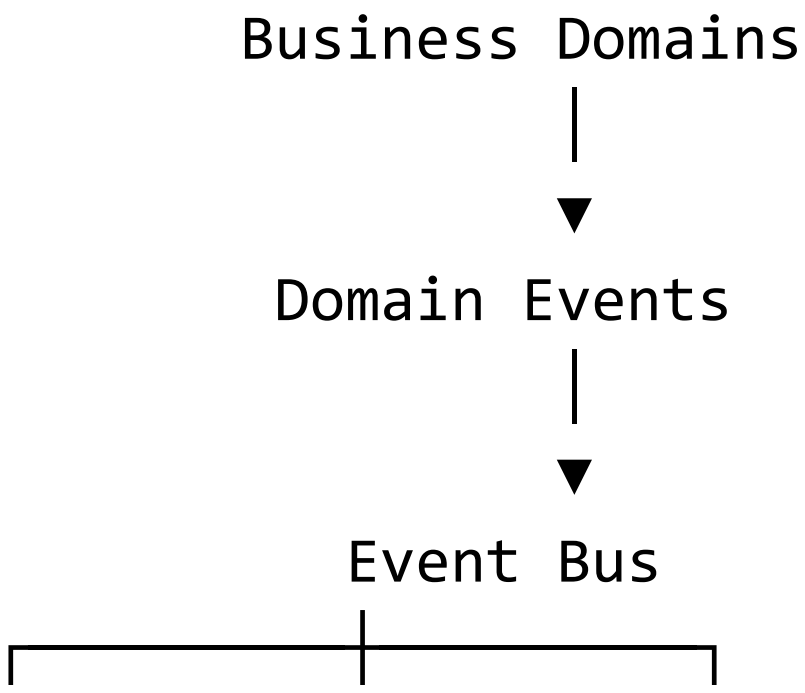
Examples include:

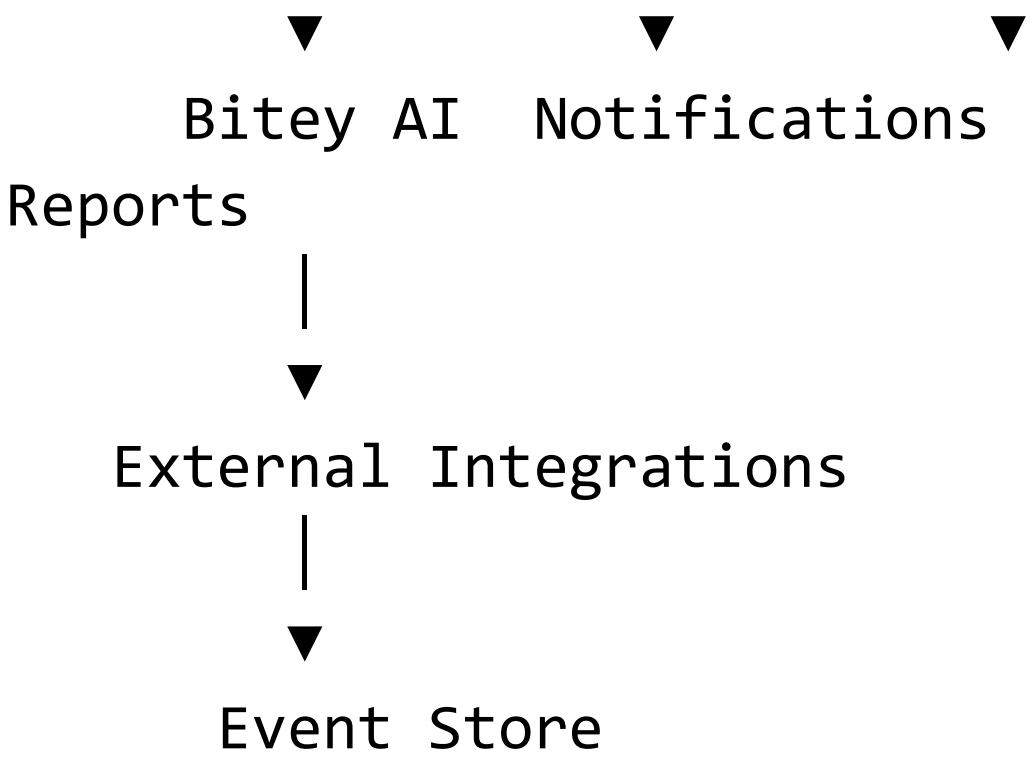
- ERP integration

- CRM integration
- Accounting systems
- Business Intelligence
- Machine Learning pipelines
- Mobile notifications
- IoT device monitoring

All become consumers of the same event ecosystem.

Canonical Architecture





Architecture Principles

The Event-Driven layer follows these principles:

- Publish facts, not commands.
- Events are immutable.
- Producers are unaware of consumers.
- Consumers are idempotent.
- Event contracts are versioned.

- Replay is always possible.
- Monitoring is mandatory.